codigo

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Aqui está a análise passo a passo do código fornecido no arquivo `codigo.py`:

Análise Passo a Passo

1. Importação de Módulos

Python

```
aqgqzxkfjzbdnhz = __import__('base64')
wogyjaaijwqbpxe = __import__('zlib')
```

O que faz: O script importa dinamicamente os módulos padrão do Python `base64` e `zlib` utilizando a função interna `__import__()` em vez de declarações `import` tradicionais. Essa técnica costuma ser empregada para dificultar a análise estática automatizada e ocultar dependências evidentes.

2. Definição da Chave e Função de Descriptografia

Python

```
idzextbcjbgkdih = 134
qyrrhmmwrhaknyf = lambda dfhulxliqohxamy, osatiehltgdbqxk: bytes([wtqiceobrebqs
```

O que faz:

Define uma constante numérica (`134`), que atua como chave para uma operação de criptografia/descriptografia baseada em **XOR**.

Cria uma função anônima (`lambda`) que percorre uma sequência de bytes e aplica o operador XOR (`^`) bit a bit utilizando a chave definida.

3. Dados Ofuscados (`Payload`)

```python
lzcdrtfxyqiplpd = 'eNq9W19z3MaRTyzJPrmiy93VPSSvqbr44V4iUZZkSaS+xe... [truncado]
```

O que faz: Armazena uma longa string codificada contendo dados altamente compactados e ofuscados. Esta string representa o corpo principal do script (o *payload* verdadeiro), que permanece oculto no código-fonte visível.

4. Descompactação, Descriptografia e Execução Dinâmica

```python
runzmcxgusiurqv = wogyjaaijwqbpxe.decompress(aqgqzxkfjzbdnhz.b64decode(lzcdrtf
ycqljtcxxkyiplo = qyrrhmmwrhaknyf(runzmcxgusiurqv, idzextbcjbgkdih)
exec(compile(ycqljtcxxkyiplo, '<>', 'exec'))
```

O que faz:

1. **Decodificação Base64:** Converte a string criptografada de texto para bytes originais.
2. **Descompressão Zlib:** Descompacta os dados comprimidos.
3. **Descriptografia XOR:** Aplica a chave `134` para reverter a ofuscação XOR.
4. **Execução Dinâmica (`exec`):** Compila e executa o código final diretamente na memória do interpretador Python.

Veredito Final

Este arquivo é um malware (especificamente, um carregador / *dropper* ou *stub* malicioso).

Justificativa:
Nenhum software ou script legítimo e benigno emprega camadas múltiplas de ofuscação (Base64 + compressão Zlib + criptografia XOR) combinadas com execução dinâmica em memória (`exec`) sem um motivo estritamente voltado para burlar a detecção de antivírus, ocultar cargas úteis maliciosas e impedir a inspeção humana do código executado. Este padrão é característico de ferramentas de acesso remoto (RATs), *stealers* ou *stubs* de vírus/malwares.

Qual é a categoria principal deste malware?

A categoria principal deste malware é **Dropper** (ou **Loader** / Carregador).

Justificativa Técnica

Função de Carregamento (*Stager*): O script não contém a lógica maliciosa final de forma legível; o seu único objetivo é decodificar, descomprimir e desencriptar a carga útil (*payload*) oculta na string `lzcdrtfxyqiplpd` .

Ofuscação em Múltiplas Camadas: Utiliza a codificação Base64, compressão `zlib` e uma cifra XOR (com a chave numérica `134`) para iludir a análise estática de ferramentas de segurança e analistas.

Execução em Memória: Através da instrução `exec(compile(...))` , o script executa o código final diretamente na memória RAM do sistema, evitando gravar ficheiros intermédios no disco e dificultando a detetar por soluções antivírus tradicionais.

Este tipo de formato é habitualmente empregue como o vetor inicial para entregar ameaças mais perigosas, tais como *infostealers* (roubadores de credenciais), *RATs* (Remote Access Trojans) ou *malware* de acesso remoto.